iPRA without supplied introduction witnesses

Three inference families and a smaller complete checker

Prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

An existential introduction need not receive a witness term. In the presentation developed here it always uses the newest available numerical parameter, or zero when no parameter is available. A gated naming construction recovers introduction at every arbitrary term. The complete calculus has three inference families, one universal-reflexivity clause, one numerical recursor, and equations and a single paired binder as its formula constructors. Its ordinary realization is bi-interpretable with full first-order intuitionistic primitive recursive arithmetic, with whole numerical domains, actual equality, both numerical composites given by $x = y$, both formula round trips, and equiconsistency.

The standalone checker is 1,925 normalized bytes, 630 significant Python tokens, and 701 AST nodes. These are all below the corresponding attained coordinates of the inspected recent strict implementations: 1,990 bytes, 636 tokens, and 708 nodes, belonging to different programs. A matched comparison isolates the contribution of omitting the witness field. An independent scope-completion lemma removes a redundant template validation traversal. Full normalization remains part of the trusted implementation; a bounded-fuel verifier is included. The repository's 508-byte compact profile has a different input contract and remains smaller. This is a demonstrated further reduction, not an absolute minimality theorem. The new metatheorems are written proofs, supported by executable checks, rather than a new Lean formalization.

1 Result, baseline, and the meaning of simpler

The comparison is pinned to ProveIt commit `11814771b7490dfbea0ed5fcf0f5b23586334764`, including its 73 filed research packages and the six archives then in `ConciseIPRA/docs/incoming/`. The repository's native scalar theory already has a formalized bi-interpretation with iPRA; its recent presentations have removed ordinary logical primitives, separate equality rules, and several explicit witness and formula fields [1, 4, 2, 3].

The inherited three-family presentations use clause application, paired introduction, and an elimination rule that also substitutes an equality. Some determine the substitution from the opened clause; others receive a separately proved equality. They still supply an arbitrary term at paired introduction. The change here removes that entire input: the introduction witness is fixed by the current numerical environment. This restriction is stronger than requiring a variable witness, and its completeness needs a careful proof because adjoining a fresh parameter changes the witness selected by the rule.

The second change is a checker theorem: a transport template need not be separately checked for scope and canonical form when both of its instantiated formulas are checked. A successful template has a legal canonical replacement with the same two instances. This removes a redundant pass without weakening the checks on active formulas.

The remaining ingredients are inherited or natural combinations of existing ingredients: counter-aware successor-seeded recursion, full normalization, clauses, a reflexive-zero-row introduction selector, and equality substitution at witness discharge. In particular, "three families"

alone is not new. The novelty claim is restricted to the inspected repository and archives; no priority claim over all proof-theoretic literature is intended.

Complexity contract. All reported measurements count one complete Python acceptance module: parsing, sorting, scopes, normalization, substitution, inference checking, and successful termination. Documentation is removed and the source is normalized with `ast.unparse` under CPython 3.12.14, followed by one line feed. Significant tokens exclude layout and comments; AST nodes include operator and context nodes. Gzip is informational. The reference sources and the measurement script are included. The three-coordinate minimum among a finite collection of programs is an attained comparison envelope, not a lower bound on all possible proof checkers.

What is being axiomatized. The theory is a full intuitionistic first-order arithmetic through its ordinary realization. Its native presentation is allowed a different logical vocabulary. Quantified-formula induction is never introduced. There is no axiom asserting totality of a universal primitive-recursive evaluator. Each numerical description is a fixed finite term.

2 Three instructions with no supplied introduction witness

2.1 Numerical notation and its exact reductions

The numerical syntax is 0, free and bound numerical variables, and

$$R(n, a; i, u.h),$$

where only i, u are bound in h. The defining computations are

$$R(0, a; i, u.h) = S(a), \qquad R(Sn, a; i, u.h) = h[n/i, R(n, a; i, u.h)/u], \qquad S(a) := R(0, a; i, u.0).$$

Zero-counter bodies are canonically erased: $R(0, a; i, u.h)$ is represented as $R(0, a; i, u.0)$. Terms are full computation normal forms. Every substitution below is simultaneous, capture avoiding, and followed by normalization, written N. This is a metasyntactic convention; no object-language universal evaluator or uniform normalization bound is postulated.

The following three finite macros are useful:

$$\begin{aligned} P(n) &= R(n, 0; i, u.i), \\ C(n, a, b) &= P(R(n, a; i, u.S(b))), \\ J(n, a; i, u.h) &= P\big(R(n, a; j, v.S(h[j/i, P(v)/u]))\big). \end{aligned}$$

Here j, v are fresh and substitution in the last display is simultaneous. Equivalently its update is $(j, v) \mapsto S(h(j, P(v)))$. All of the following are literal identities after normalization:

$$\begin{aligned} P(Sn) &\equiv n, & P(0) &\equiv 1, \\ C(0, a, b) &\equiv a, & C(Sn, a, b) &\equiv b, \\ J(0, a; i, u.h) &\equiv a, & J(Sn, a; i, u.h) &\equiv h[n/i, J(n, a; i, u.h)/u]. \end{aligned}$$

In particular, P is a left inverse of successor; its zero value is *one*, not zero. When a conventional predecessor is wanted, use $p(n) = C(n, 0, P(n))$. It has $p(0) \equiv 0$ and $p(Sn) \equiv n$. The scalar iterator of the source calculus embeds as $I(n, a; u.h) = J(n, a; i, u.h)$ with the counter coordinate unused.

2.2 Complete logical rules

Formulas are equations and $D_{x,y}B$. A clause is either $[\,; y.B]$ or $[A; y.B]$, with y absent from its antecedent. Every derivation starts with the one fixed clause

$$\mathcal{R} = [\,; y.y = y].$$

An ordinary local assumption A is the unconditional constant clause $[\,; v.A]$. The ordinary readings of D and a conditional clause are

$$D_{x,y}B \;:=\; \exists x(B(x,0) \to \forall y B(x,y)), \qquad [A; y.B] \;:=\; A \to \forall y B.$$

An environment orders its free numerical parameters. Let ℓ be its newest parameter, and let $\ell = 0$ in an empty environment. For a matrix B define $B_r(y) = N(B(r,y))$. The three instruction families are:

1. **Use**: select a retained clause and a well-scoped term t; prove its antecedent when it has one; conclude its normalized body at t.
2. **Introduce**: conclude $D_{x,y}B$ from

$$\Gamma, B_\ell(\eta) \vdash B_\ell(Sz), \qquad \eta = \begin{cases} z, & B_\ell(0) \text{ is literally an equation } r = r, \\ 0, & \text{otherwise,} \end{cases} \tag{I}$$

   where z is fresh. The selected witness is the newest *outer* parameter; the new z is available only in the premise. No witness, induction mode, or base proof is supplied in the instruction.
3. **Substitute and discharge**: prove a major $D_{x,y}B$ and open its ordinary clause at a fresh parameter b:

$$\mathcal{C}_B(b) = [N(B(b,0)); y.N(B(b,y))].$$

   In that same extended context prove $s = t$ and $N(F(s))$, for a supplied one-place native formula template F. Conclude $N(F(t))$, provided that this conclusion contains no free b:

$$\frac{\Gamma \vdash D_{x,y}B \quad \Gamma, \mathcal{C}_B(b) \vdash s = t \quad \Gamma, \mathcal{C}_B(b) \vdash N(F(s))}{\Gamma \vdash N(F(t))}. \tag{E}$$

   The terms s, t and the template may use b. The final normalized formula may not. The template's placeholder is separate from all numerical binders; substitution into it is capture avoiding.

There are no other logical, equality, arithmetic, case, induction, or structural rules. The computations stated above belong to canonical syntax. The designated contradiction is $0 = S0$.

2.3 Soundness and the role of the selector

Use is ordinary clause application. Rule (E) is ordinary existential elimination followed by equality substitution in its branch; the condition on the final formula is exactly its eigenparameter condition. This remains sound when a substitution target contains b.

For (I), a nonreflexive zero row selects cases, and numerical coverage combines the retained zero row with the supplied successor derivation. A reflexive zero row selects progress, but only for a literal equation; contextual equation induction proves all its instances. Its zero instance is a reflexivity theorem. In either case ℓ supplies a legal existential witness. Thus every rule is sound in ordinary intuitionistic arithmetic with coverage, separation, and equation induction. In particular the selector introduces no induction on quantified formulas.

At an allowed witness, a case premise can always be used. If the selector instead requests progress, its discarded zero-row assumption is literally reflexive. Supply that theorem in place of the old assumption and ignore the new induction assumption. In the constructive proof below this is used as an operation on uniformly supplied branch builders, not as an unproved parameter-weakening principle for arbitrary target proof trees.

2.4 Preparatory constructions independent of equality transport

Write $q_y A = D_{w,y} A$ with w unused. Its introduction uses an unused witness, so changing which outer parameter is newest does not change its matrix instances. Its ordinary elimination follows from E by using $s = t = 0$ and the constant template whose value is the wanted conclusion. The equality premise is Use of $\mathcal{R}$ at 0.

Consequently the ordinary case-introduction and elimination rules for q are constructible from I, E, and Use. In a case introduction, if its zero row is reflexive, use the preceding zero-row observation. This gives numerical cases for all native formulas, with branch proofs constructed in their exact environments. Neither arithmetic equality transport nor equation induction is needed for these logical case constructions.

For completeness, formula gates and padding are likewise available before transport. Gate every numerical side of the bottom equation in A by $C(v, 0, -)$, leaving its D binders intact; call the result $G_v A$. Then $G_{Sz} A \equiv A$, while $G_0 A$ is a finite tower of D binders over $0 = 0$, and is uniformly provable by I and reflexivity. For persistent premise builders, the padding $L(A) = q_y G_y A$ is interderivable with A: one direction ignores its local base and uses A, and the other uses the closure at 1 with the uniformly provable zero row. Padding to the same binder depth allows the usual componentwise selector of two formulas. Thus implication is $q_y \operatorname{sel}(y, A, B)$, up to this harmless padding, with its ordinary introduction and elimination constructions. This paragraph describes uniform proof-building operations; arbitrary target cut or parameter weakening is only recovered after the main equivalence theorem. Every introduction used here has an unused witness, except introductions of the all-zero truth towers, whose result is independent of that witness.

2.5 An equality-free name for every numerical term

Define

$$\operatorname{Name}(t) = D_{x,u}\big(C(u, 0, x) = C(u, 0, t)\big).$$

Ordinary elimination at a fresh b supplies $b = t$ by Use at 1; the needed antecedent is literally $0 = 0$. No equality reasoning is involved in this extraction.

Put

$$F(y) = D_{x,u}\big(C(y, 0, C(u, 0, x)) = C(y, 0, C(u, 0, P(y)))\big).$$

Its zero instance is a tower over $0 = 0$, so I at whichever outer parameter is newest proves $F(0)$. In the successor branch, z is the newest parameter. Introduce the inner D with the automatically selected witness z. Its zero row is $0 = 0$ and its successor row is

$$z = P(Sz) \equiv z = z.$$

Use of $\mathcal{R}$ proves this row, ignoring the local hypothesis. Numerical cases therefore prove $F(S(t))$. All computations are literal: $P(S(t)) \equiv t$ and $C(S(t), a, b) \equiv b$. The resulting formula is exactly $\operatorname{Name}(t)$.

This proof is reconstructed in each ambient environment. Its important features are that it uses no arbitrary introduction witness, no equality substitution, and no predecessor theorem. The shift by $S(t)$ is necessary; the zero value $P(0) = 1$ is erased by the outer gate.

2.6 Translation from the diagonal native kernel

Use the established native diagonal comparison calculus as source. Its terms include the scalar iterator, it has ordinary equality transport, closure use and witness elimination, and its D introduction has an arbitrary supplied witness. Its progressive matrices have the literal shape

$$I(y, y; u.h) = I(y, y; v.k).$$

All other introductions are by cases. The repository proves this source equivalent to the unrestricted native calculus, and hence to its ordinary iPRA realization. Map its scalar iterator to J with the counter unused, map its equations and binders compositionally, and normalize numerical subterms. A progressive diagonal matrix always has the literal normalized zero row $0 = 0$, independently of its witness and its update bodies.

The translation is a relative induction on source derivations. Its induction statement allows an arbitrary target ambient environment and context, with persistent builders for the source assumptions. A builder can be replayed after adjoining fresh parameters and clauses. At the root an assumption builder is hypothesis use. Every replacement of a local source assumption below uses only hypothesis use, reflexivity, clause use, and transport through a dummy major

$$D_{x,y}(0 = 0).$$

The dummy major's introduction is independent of its selected witness, so these builders remain persistent. This invariant replaces the otherwise invalid shortcut of weakening arbitrary target proofs after the newest parameter has changed.

Computation axioms translate to reflexivity, because their two sides have the same normal form. Hypothesis use invokes its persistent builder. Ordinary source witness elimination is an E step with a constant template and the reflexive equality $0 = 0$. Its local source assumption $Q_y B(b, y)$ is represented by the target clause $[B(b, 0); y.B(b, y)]$. Reify that clause as the native formula $q_y B(b, y)$ by an unused-witness I instruction: in its branch use the retained clause at Sz, supplying its zero antecedent from the local case assumption, or from reflexivity if the selector chose progress. This reification is a persistent builder because its selected witness is unused. It supplies exactly the local-assumption realization required by the induction hypothesis for the source elimination branch.

For a source Q elimination, translate its major to $q_y B(y)$ and open that formula first. Only then invoke the induction hypothesis on the strictly smaller source proof of $B(0)$, in the extended target context. Use its resulting proof as the antecedent of the newly retained clause, specialize at the requested term, and close E with a constant template and $0 = 0$. One must not translate the base premise outside the opening and then silently weaken the resulting target proof tree into its branch.

For a source equality-transport node, first open the dummy major above. Now apply the induction hypothesis to its two strictly smaller source premises in that extended context, and finish E with their prescribed formula template. If the source uses the crossed one-term convention, derive symmetry through the same dummy construction, or use the equivalent full-transport source presentation. It would be circular at this stage to claim arbitrary target parameter weakening in order to move already translated premise trees into the dummy branch; reapplying the induction hypothesis is the correct construction.

For a source introduction at witness t, first prove $\mathrm{Name}(t)$ and open it at a fresh b. The branch receives the persistent equality $b = t$, and b is now the newest outer parameter. The target I instruction therefore selects exactly b.

If the source introduction is progressive, its matrix is diagonal. Its normalized zero row is $0 = 0$ even at b, so target I also selects progress. Transport the target assumption at b to the source assumption at t, use the induction hypothesis on the smaller source branch, and transport its result back to b.

If the source introduction is by cases and target I chooses cases, perform the same two transports on the zero-row assumption and successor conclusion. If target I chooses progress instead, its zero row at b is reflexive. Obtain that row by Use, transport it to the zero row at t, use the induction hypothesis on the smaller source branch, and transport back. The extra target progress assumption is unused. For the final transport back, open its dummy major before invoking the induction hypothesis on the source branch; the recursive call is made in that actual further-extended context. The builders for its assumptions remain persistent there. Thus no

completed target branch proof is moved into a new parameter scope by an unproved weakening step.

In every case I produces the unchanged major formula $D_{x,y}B$, which does not contain b. Close its naming elimination with a constant template and $0 = 0$. Every recursive translation call is on a strictly smaller source derivation; there is no appeal to the node being replaced.

Theorem 2.1 (Consequence preservation)**.** *The new three-instruction calculus proves exactly the normalized translations of the native diagonal kernel, with the ordinary logical and arithmetic meanings described above. The transformation preserves open assumptions through their stated realizations, and in particular preserves closed theorems and the contradiction* $0 = S0$*.*

For target formulas using R essentially, extend the comparison language by that fixed primitive-recursive definition before applying the argument. This is a definitional extension of the ordinary iPRA realization. The logical comparison and the diagonal argument are parametric in additional numerical terms: full formula transport is already permitted, and the only special numerical identities used by them are those of $0, S, I$. The additional R computation axioms again translate to reflexivity. This accounts for the whole target language, not merely its scalar-iterator image. The written parametric argument is not a claim that the unchanged Lean files have already formalized the enlarged syntax.

2.7 Whole-domain interpretations and both numerical composites

Let T_R be ordinary intuitionistic first-order equality logic over the staged R descriptions, with their displayed computations, numerical coverage, $(0 = S0) \to \bot$, and contextual induction for literal equations. The native logical comparison supplies its complete proof presentation. The explicit falsity bridge in this ordinary realization is essential; the equation $0 = S0$ is the native contradiction, not by itself an ordinary logical falsum in a positive singleton structure.

Interpret R in iPRA by its fixed primitive-recursive equations. Conversely, initial functions and composition translate directly, and a fixed primitive recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p})$$

translates to

$$f^*(n, \vec{p}) = J(n, g^*(\vec{p}); i, u.h^*(i, u, \vec{p})).$$

There is no pairing construction and no universal evaluator here: the counter coordinate is an explicit bound argument of each fixed update. The ordinary predecessor p, selector C, addition and monus have their usual equations. The inherited equality-reflection argument from coverage, separation and literal-equation induction then yields every quantifier-free induction instance. Successor injectivity follows immediately by applying P; successor separation follows by the selector's zero/successor test.

For two fixed recursive definitions with identical initial values and provably identical updates, literal-equation induction on equality of their values proves uniqueness at every internal counter. Apply this internally while inducting externally on each finite description: a double-translated description has the same equations as its original, and hence the same values. This argument works for both the R and ordinary primitive-recursive descriptions, with every passive parameter fixed during each induction.

Both interpretation domains are the entire numerical sort, defined by $x = x$, and equality is actual equality. The two numerical comparison isomorphisms are therefore both the identity graph $x = y$. The displayed meanings of D, clauses, and the derived logical connectives give the two formula round trips by external induction on formula syntax. Preservation of $0 = S0$ and ordinary falsity gives effective proof translations of contradictions in both directions and therefore equiconsistency.

These are internal uniqueness and formula-equivalence arguments, not an inference from agreement in the standard natural numbers. No claim of an absolute minimal presentation or a new Lean verification is made by this proof alone.

3 Normalization and the arithmetic comparison

3.1 A precise canonical syntax

For the metatheory, first distinguish the raw constructors $0, S, R$. Orient the two recursor equations as

$$R(0, a; i, u.h) \longrightarrow Sa, \qquad R(Sn, a; i, u.h) \longrightarrow h[n/i, R(n, a; i, u.h)/u].$$

Reductions may occur in every numerical subterm, including an update body under its two binders. The mathematical normal form is taken in this raw syntax. Its finite encoding then prints Sa as $[0, a, 0]$ and an unreduced $R(n, a; i, u.h)$ as $[n, a, h]$. Thus the implementation's zero-counter clause is a canonical constructor operation, not a rewrite of $R(0, a; 0)$ to itself to be repeated forever.

Lemma 3.1 (Canonical computation)**.** *Every well-sorted finite term has a unique computation normal form. Reduction introduces no new free variable. For every capture-avoiding substitution* σ*,*

$$N(t[\sigma]) = N(N(t)[N \circ \sigma]).$$

The corresponding statements hold for formulas and one-place templates.

Proof. One standard strong-normalization proof embeds a raw recursor into the ground-type natural-number recursor: its starting value is Sa, and its step is the finite function $(i, u) \mapsto h(i, u)$. Variables in h are bound numerical arguments; there is no application of a variable as a function. The usual reducibility proof specializes directly to this fragment. Its recursor case inducts on a computable counter and its reducts: at zero the result is the computable starting value, and at a successor the recursive call is computable at the predecessor and the computable update preserves computability. At a neutral counter the recursor is neutral and its reductions are those of computable proper subterms. Structural induction on a finite term, with computable terms substituted for its numerical variables, establishes computability and hence strong normalization. Applying the same argument under fresh variables handles reductions inside update bodies.

The two root patterns are disjoint and left-linear. Nested reductions commute by the capture-avoiding substitution lemma; duplicating an update or an argument duplicates its reducts on the other branch of the diagram. The parallel-reduction argument therefore gives confluence. Strong normalization and confluence give a unique normal form. Applying a substitution to each computation equation yields another valid computation; normalizing the two sides of this conversion proves the displayed coherence identity. All right sides use only variables from their left sides, proving the free-variable assertion. □

This is an external metatheorem about finite expressions. There is no claim that normalization of arbitrary supplied descriptions admits a uniform primitive-recursive bound. In particular, a full normalizer is not free infrastructure in the complexity comparison: its whole source is included, and the bounded verifier charges every recursive traversal.

3.2 Computation is also provable in the ordinary theory

The ordinary realization T_R mentioned above uses raw finite recursor descriptions. An instance nested in a counter or initializer is transported by equality. A reduction inside an update is justified by the following body-extensionality lemma, rather than by silently treating a bound function as a first-order numerical argument.

Lemma 3.2 (Extensionality for a fixed update)**.** *If $h(i,u) = k(i,u)$ is derivable uniformly in its two displayed variables, then*

$$R(n,a;i,u.h) = R(n,a;i,u.k)$$

is derivable for every n, with all passive parameters fixed.

Proof. Use literal-equation induction on the displayed equality. At zero both sides are Sa. At a successor, computation exposes the two updates. The induction equation first replaces the old state in one update; the supplied pointwise update identity then changes h into k. All inductions used here have a literal equation as their formula. □

Consequently each finite reduction to $N(t)$ yields an internal proof of $t = N(t)$. An external induction on a formula gives the corresponding equivalence between its raw and normalized readings. There is no use of a uniform internal truth or evaluation predicate.

3.3 The full comparison is over finite descriptions

The comparison theory for the proof translation is the diagonal native calculus, conservatively supplied with fixed R descriptions and their computation equations. Each occurrence of $R(n,a;i,u.h)$ first compiles the proper subdescriptions of a, h and then introduces the primitive recursion with that fixed update. The resulting description is finite. This construction is staged by syntax, not by a runtime code variable.

The diagonal proof works uniformly in the enlarged numerical vocabulary: its selector identity and its equation-induction simulation treat the two sides of an equation as arbitrary term templates. The new computations translate to literal reflexivity after normalization. Conversely, iPRA interprets every such fixed description by primitive recursion. This supplies the whole target language, including equations with R in their updates, to the source-to-target argument.

For clarity, the forward compilation of an ordinary primitive-recursive definition is explicit:

$$\begin{aligned} 0^* &= 0, & S^*(x) &= R(0,x;i,u.0), \quad \pi_j^*(\vec{x}) = x_j, \\ (f \circ \vec{g})^*(\vec{x}) &= f^*(g_1^*(\vec{x}), \dots, g_m^*(\vec{x})), \\ (\mathrm{rec}(g,h))^*(n,\vec{x}) &= P\big(R(n, g^*(\vec{x}); i, v.S(h^*(i,P(v),\vec{x})))\big). \end{aligned}$$

The reverse compilation sends $R(n,a;i,u.h)$ to the fixed ordinary primitive-recursive function with base $S(a)$ and step h. These are translations of descriptions; no quantification over functions is added.

Theorem 3.3 (Bi-interpretability and consistency)**.** *The ordinary realization of the three-instruction calculus is bi-interpretable with* iPRA*. Both domains are the full numerical sort, equality is genuine equality, and both numerical comparison maps have graph $x = y$. Each full formula is provably equivalent to its double translation. In particular the new calculus proves $0 = S0$ if and only if* iPRA *proves $\bot$.*

Proof. The rule-by-rule argument establishes the presentation theorem for T_R. The interpretation of its calculations in iPRA is the fixed recursion just described. Coverage and separation hold in iPRA, and literal-equation induction is an instance of quantifier-free induction. For the reverse direction the displayed compilation validates the primitive-recursive equations; applying P proves successor injectivity. If $Sx = 0$, the selector equations force $0 = S0$, so successor separation also holds. The scalar iterator J supplies the already established scalar equality-reflection construction. Thus every quantifier-free formula is equivalent to the vanishing of a fixed numerical term, and its induction instance follows from equation induction. The attributed full equality-reflection derivation is reproduced in the reference folder [5].

For a fixed recursive definition, equation induction proves uniqueness from its base and step equations. External induction on each finite description therefore proves that its double translation has the same value as the original, internally for every numerical argument and every passive parameter. The domain interpretations are unrestricted and the isomorphism is the identity. The native logical encodings below and the ordinary reading of D supply both formula round trips. Falsehood is carried to the fixed equation $0 = S0$, and its ordinary reading implies $\bot$ by the explicit separation axiom. Hence contradiction proofs translate both ways. □

The argument asserts effective finite proof translations and internal round-trip identities. It does not assert a uniform primitive-recursive normalization algorithm, a proof of consistency inside iPRA, or a new formal verification of the metatheory.

3.4 Recovering the ordinary logical vocabulary

Here is an explicit account of the logical language being recovered. Write $q_y A = D_{d,y} A$ with d unused. Formula padding is $L(A) = q_y G_y A$, where G gates the final equation by C. Padding is equivalent to A; it permits two formulas to be brought to the same number of paired binders. After freshening and aligning those binders, replace their two final equation endpoints componentwise by the selector $C(v, -, -)$. The resulting formula $\mathrm{sel}(v, A, B)$ is equivalent to A at zero and B at a successor. All temporary bound variables are distinct from v and the free variables of A, B.

Let par be the fixed primitive-recursive parity term. One standard native encoding is

$$\begin{aligned}
\bot^* &= (0 = S0),\\
(A \to B)^* &= q_y\, \mathrm{sel}(y, A^*, B^*),\\
(A \wedge B)^* &= q_y G_y\, \mathrm{sel}(\mathrm{par}(y), A^*, B^*),\\
(\exists x\, A)^* &= D_{x,y} G_y A^*,\\
(A \vee B)^* &= \exists v\ \mathrm{sel}(v, A^*, B^*),\\
(\forall x\, A)^* &= A^*(0) \wedge q_x A^*(x).
\end{aligned}$$

The last three displays use the macros defined in earlier lines. Bound variables in each line are fresh. Selecting 1 and 2 in the conjunction gives its two components. For implication, its zero row is the antecedent and each successor row is the consequent. Existential introduction and elimination are exactly the gated paired rules. Numerical coverage proves the disjunction case rule. Transport along $0 = S0$ in a gate changes its provable zero instance into any chosen formula, giving explosion. These are intuitionistic derivations; no excluded middle for formulas with quantifiers is used.

The diagonal native equivalence gives these rules under arbitrary retained assumptions, not just for closed arithmetic identities. By structural induction the ordinary reading of each macro is equivalent to its intended connective. Expanding the ordinary reading of a native D back through the macros gives the original D, again under arbitrary assumptions. Together with the numerical uniqueness argument this proves both formula round trips, rather than only mutual interpretability of theorem sets.

4 Checking a template through its two instances

The transport part of discharge supplies a formula template $F(v)$, two terms s, t, and checked proofs of $s = t$ and $N(F(s))$. Its proposed result is $N(F(t))$, which must equal the already checked outer conclusion. A separate traversal checking the scope and normality of F is redundant. This statement concerns only this template: all actual equations and conclusion formulas remain sorted, scoped, and normal-form checked.

Lemma 4.1 (Completion of an accepted template)**.** *Suppose the generic substitution traversal of a finite JSON expression F succeeds at both checked numerical terms s, t. Suppose the two resulting normalized expressions are well-scoped formulas in the local numerical environment (the second may satisfy the stricter outer scope). Then there is a well-scoped canonical formula template $F'(v)$, using only that environment and its one formal parameter v, such that*

$$N(F'(s)) = N(F(s)), \qquad N(F'(t)) = N(F(t))$$

as literal syntax.

Proof. First consider sorting. At every proper child the generic traversal uses the exact expected numerical or formula sort, before performing any erasing reduction. At its root it may temporarily accept an unclassified list. A root of length other than one, two, or three does not reduce and cannot produce a checked formula. A numerical root, including a recursor or an integer replaced by a numerical term, stays numerical under every reduction. Thus the successful checked formula instance forces the original root to be an equation or a paired formula. The input is consequently a sorted formula template, possibly with extra numerical parameters or reducible numerical subterms.

Interpret every negative level outside the allowed local range as an additional free numerical parameter. At binder depth d, interpret a positive index larger than $d + 1$ in the template as another free numerical parameter: the one allowed index above the local binders is the formal parameter v. This interpretation is consistent under traversing a binder because the excess index is measured relative to its depth. There are finitely many additional parameters in F.

Let ρ replace all of these additional free parameters by 0, fixing v, the permitted local parameters, and every bound variable. Neither s nor t contains an additional parameter. Capture-avoiding substitution and normalization coherence therefore give, for $u = s, t$,

$$N((\rho F)[u/v]) = N(\rho(F[u/v])) = N(\rho N(F[u/v])) = N(F[u/v]).$$

The last equality holds because the checked endpoint contains no additional parameter and is already normal. In particular, allowing a new reduction after closing an extra parameter does not invalidate the argument; coherence, rather than a claim about identical reduction traces, is being used.

Finally set $F' = N(\rho F)$. It is sorted, well scoped, and canonical. A second application of normalization–substitution coherence gives the two required literal identities. The one-hole template may contain zero, one, or several occurrences of v; none of the argument requires linearity. □

Replacing the supplied template by F' leaves the two premises and the exported conclusion unchanged. Hence every successfully checked use of the relaxed interface is an instance of ordinary equality transport. Conversely, every previously checked canonical template is still accepted. This removes one trusted traversal without changing the consequence relation or permitting an out-of-scope active formula.

The equation premise must be checked before the template is substituted, so that s, t are known to be numerical terms. The branch instance and the outer conclusion must both retain their existing checks. Omitting one of these conditions is not justified by this lemma.

5 The complete certificate checker

5.1 Finite representation and scoping

The file `code/checker.py` is the complete trusted program. It uses only Python's standard JSON module. Input is a two-element JSON array `[goal, proof]`. Acceptance means a zero exit

status and the line `ACCEPT`; an exception, nonzero exit, or exhausted resource is not acceptance. The `assert` statement is not used.

Zero is integer 0. Negative integers are free numerical parameters. Positive integers are de Bruijn bound indices, starting at 1. A term is an integer or a triple `[n,a,h]`; the last child binds two variables, with the state at index 1 and counter at index 2. The canonical successor is `[0,a,0]`. A formula is `[s,t]` or `[B]`, the latter binding the inner variable y at index 1 and witness variable x at index 2.

The initial context is the single unconditional clause `([], [1,1])`. Every scope-extending instruction appends one clause and introduces exactly one fresh numerical parameter. Thus for a context of length n, available parameters are $-1, \ldots, -(n-1)$, the fresh parameter is $-n$, and the automatically selected introduction witness is $1-n$. At the root it is 0. Clause indices are zero based. Booleans are not accepted as integers.

Both the recursor and D bind two variables in their final child, whereas an equation binds none. One traversal therefore suffices for terms, formulas, scope checks, substitution, shifting, and normalization. Children are checked before a computation can erase them.

5.2 The three proof instructions

Packet	Meaning
`['u',j,t,p]`	Use clause j at t; p proves its antecedent when it has one.
`['i',p]`	Introduce the goal's paired binder with its fixed environmental witness; p proves the required successor instance.
`['e',B,F,s,t,p,q,r]`	p proves DB; after opening at a fresh name, q proves $s = t$, and r proves $N(F(s))$; export $N(F(t))$.

The placeholder of F is its one extra de Bruijn variable. The equality premise is checked before either substituted instance. The second instance must equal the outer goal, whose full syntax and scope were already checked. All active goals are normal forms. The payload p of an unconditional Use is deliberately unused; the examples put `null` there. This is the absence of a premise, not an accepted unverified derivation.

Two substitutions, in the right order. To reduce the successor recursor, the checker first substitutes the predecessor counter into the *outer* of its two body slots, while protecting the inner state slot. It then substitutes the predecessor state into the inner slot:

$$\mathrm{sub}\big(\mathrm{sub}(h, c, 1), N(R(c, a; i, u.h))\big).$$

The first substitution shifts the inserted counter across the still-bound state variable; the second removes that slot again. This works when the counter and predecessor state themselves refer to surrounding binders. Substituting the state first without a compensating shift would capture those variables. A separate simultaneous-substitution implementation and a named-variable oracle check this distinction.

5.3 A bounded certificate relation

The companion `code/checker_fuel.py` takes `[goal,proof,fuel]`. It decrements a nonnegative integer budget on each call to the common traversal and to the inference checker, rejecting when the budget is exhausted. Every finite successful run of the main program has a sufficient finite budget, so the two interfaces have the same theorems. The bounded verifier supplies a primitive-recursive finite-certificate relation; there is no need to assume that an arbitrary

normalization completes within a uniform primitive-recursive bound. The budget and its checking code are included in that program's separate measurements. Concrete Python recursion and memory limits still bound the certificates a particular process can execute.

5.4 What was verified

The accompanying scripts generate exact certificates and replay the standalone modules. They include the defining computations, capture-safe substitution under both binder kinds, full formula equality transport, an induction proof that uses its step hypothesis, and names for nonzero terms whose introductions contain no witness fields. Negative examples test escaping eigenparameters, a forged equality premise, false arithmetic, illegal clauses and indices, wrong predecessor slots, malformed syntax, and missing induction assumptions. Runs with `python -O` retain the same acceptance checks.

The independent normalization audit uses a named-variable implementation and a numerical evaluator, rather than calling the checker's substitution to produce its expected answers. A separate template-completion experiment samples 5,000 raw sorted templates; 451 have two valid instances, and 109 of those contain extra parameters erased by normalization. All checked instances agree with their legal canonical repairs. These finite checks support the implementation and catch concrete binding errors. The general soundness, completeness, and bi-interpretation claims rest on the written proofs above.

The exact counts, hashes, subprocess results, and independently computed metrics are recorded in `evidence/`. The unchanged source Lean files are provided for attribution and inspection; they are not a newly built Lean project for this three-instruction calculus.

6 Reproduced comparisons and limits

Complete standalone module	Bytes	Tokens	AST	Gzip-9
New, no witness field	**1925**	**630**	**701**	778
Matched explicit-witness control	1967	636	707	790
New, separate template validation	1985	646	717	792
New, finite fuel	2056	667	748	826
Matched explicit-witness, finite fuel	2098	673	754	838
Incoming explicit Name	2010	636	708	791
Incoming inferred opening	1990	683	771	819

The matched explicit-witness control has the same recursor, normalizer, clauses, discharge rule, template-completion treatment, and input contract. Only its introduction packet supplies and validates a witness term. Removing that field saves 42 bytes, six tokens, and six AST nodes. Equivalently, the main module is approximately 2.14% smaller in normalized bytes than this tightly matched control. The stronger mathematical restriction on introductions is proved complete above.

Removing the separate template-validation pass saves a further 60 bytes, 16 tokens, and 16 AST nodes relative to the corresponding fixed-witness version that retains the pass. The underlying mathematical rule is still ordinary equality substitution; the completion lemma justifies the more permissive auxiliary-template representation.

Against the two selected incoming strict modules, the new implementation has fewer bytes, tokens, and AST nodes in each comparison. Those comparisons also change the recursor and shared binding traversal, so the whole improvement must not be attributed to witness removal alone.

Separate compact profiles in the inspected corpus	Bytes	Tokens	AST
Incoming compact byte profile	508	267	302
Existing compact token profile	594	256	283
Incoming compact AST profile	580	263	276
Incoming native public-goal profile	780	360	409

These programs remain smaller. The first three use the corpus's documented integer/list and guarded-public-goal contract, with surplus raw syntax given a separate interpretation; they are not independent validators of the complete public grammar. The last retains its native public-goal validator, but its internal proof protocol is still different from the strict clause modules. No row is a measurement of the new theory. The numbers 508, 256, and 276 belong to different programs and must not be combined into a fictitious single checker.

The result therefore establishes a smaller complete straightforward implementation in the inspected strict family, and a genuinely smaller introduction interface with exact consequence preservation. It does not settle the unrestricted optimization problem suggested by the word "simplest." There is no proven absolute lower bound here, and this package does not meet the smallest compact-program coordinates.

6.1 Verification ledger

The final reproduced implementation evidence records the following successful checks:

Check	Count
Accepted theorem certificates	13
Adversarial rejected certificates	23
Translations to explicit-witness control	13
Translations completing templates	13
Standalone normal/optimized Python runs	140
Exact and one-unit-short fuel boundaries	26
Malformed fuel budgets	8
Exact normal forms against named-variable oracle	400
Independent numerical interpretations	1200
Bound-variable openings against oracle	480
Successor contractions against oracle	320
Targeted capture controls	2

The no-witness naming certificates for 1 and 2 require respectively 1,155 and 1,345 charged entries in the supplied fuel convention. These examples test the mechanism that makes the restriction plausible: a closed root cannot select either numeral as an introduction witness, but the resulting naming theorems are nevertheless accepted.

7 Conclusions and the remaining mathematical work

The new restriction concerns how witnesses enter a proof. An introduction uses a parameter determined by its environment, and arbitrary numerical terms enter through clause specialization. The naming theorem transfers the expressive burden from existential introduction to that existing specialization operation. Counter-aware recursion makes its required predecessor identity computational, so the transfer does not presuppose the equality or induction principles being recovered.

The most delicate part is structural: deterministic newest-parameter selection invalidates a naive proof of parameter weakening by direct induction on target derivations. The proof

instead translates source derivations with persistent assumption builders, introducing auxiliary scopes before recursively translating the premises. Ordinary weakening and substitution become admissible consequences of the equivalence.

The next substantive verification task is to formalize that relative translation and the template-completion lemma in the repository's Lean development. The current package supplies complete specifications, written proofs, executable checkers, matched controls, and reproducible finite evidence. It does not label the new theory as already Lean-checked.

A The reference notion of iPRA

The arithmetic language has a symbol for every finite primitive-recursive description: zero, successor, projections, composition, and primitive recursion. Its nonlogical axiom schemas are the defining equations, successor injectivity, and $Sx = 0 \to \bot$. Its induction rule, allowing arbitrary retained side assumptions Γ, is

$$\frac{\Gamma \vdash A(0) \qquad \Gamma, A(z) \vdash A(Sz)}{\Gamma \vdash A(t)}, \quad A \text{ quantifier-free}, \quad z \notin \mathrm{FV}(\Gamma, t, \text{passive parameters of } A).$$

Quantifier-free formulas may contain all intuitionistic propositional connectives and equality. The logical system is single-conclusion intuitionistic natural deduction with the following schemas; eigenvariables are fresh for undischarged assumptions and the indicated outer conclusions:

Operation	Inferences
Assumption	From $A \in \Gamma$, infer $\Gamma \vdash A$.
Conjunction	From A, B, infer $A \wedge B$; from $A \wedge B$, infer either component.
Disjunction	From either component infer $A \vee B$; from $A \vee B$ and branches $A \vdash C$, $B \vdash C$, infer C.
Implication	From a branch $\Gamma, A \vdash B$ infer $\Gamma \vdash A \to B$; from $A \to B, A$ infer B.
Universal	From $A(z)$ with z fresh infer $\forall x A(x)$; from $\forall x A(x)$ infer $A(t)$.
Existential	From $A(t)$ infer $\exists x A(x)$; from that formula and a fresh-witness branch $A(z) \vdash C$ infer C.
Falsehood	From $\bot$ infer any formula.
Equality	Infer $t = t$; from $s = t$ and $F(s)$ infer $F(t)$ for every capture-avoiding formula template.

Contexts are reusable. Weakening and cut are admissible. This conventional system fixes the comparison target; its logical rules are derived, rather than additionally postulated, in the three-family presentation.

B Source attribution and reproducibility

The archive includes unchanged upstream sources for the two strict comparison modules, the compact frontier, the relevant Lean theorem statements, and the five scalar-foundation fragments. Their original MIT No Attribution license is retained. The long scalar predecessor construction is background, not needed for the new naming proof. In the inherited equality-reflection fragment the letter P denotes the conventional predecessor with $P(0) = 0$; in this report that function is $p(n) = C(n, 0, P(n))$, not the shorter left inverse whose value at zero is one.

Run `reproduce.py` to verify the package hashes, execute the tests, rerun the completion experiment, and reproduce the module measurements. Run `build_pdf.py` to compile this standalone LaTeX source with pdfLaTeX. The independent artifacts are evidence and proof producers; none is imported by the standalone acceptance module.

C Complete normalized standalone module

The following listing is the entire measured 1,925-byte module. The editable source in `code/checker.py` also contains documentation.

```
import json

def need(condition):
    if not condition:
        raise ValueError('invalid certificate')

def walk(a, f, d=0, form=None):
    if type(a) is int:
        need(not form)
        return f(a, d)
    need(type(a) is list)
    size = len(a)
    need(form is None or size in ((1, 2) if form else (3,)))
    a = [walk(b, f, d + 2 * (size != 2 and j == size - 1), size == 1) for j, b in enumerate(a)]
    if size == 3:
        c, b, h = a
        if c == 0:
            return [0, b, 0]
        if type(c) is list and c[0] == 0:
            return sub(sub(h, c[1], 1), walk([c[1], b, h], lambda i, d: i))
    return a

def syntax(a, fresh, form=False):
    need(walk(a, lambda i, d: need(fresh < i <= d) or i, form=form) == a)

def sub(a, t, depth=0):
    return walk(a, lambda i, d: walk(t, lambda j, e: j + d if j > e else j) if i == d + 1 else i
     - (i > d + 1), depth)

def prove(a, p, g):
    fresh = -len(g)
    syntax(a, fresh, True)
    match p:
        case ['u', j, t, premise]:
            need(type(j) is int and 0 <= j < len(g))
            syntax(t, fresh)
            base, body = g[j]
            need(a == sub(body, t))
            if base:
                prove(base, premise, g)
        case ['e', body, template, s, t, major, eq, branch]:
            prove([body], major, g)
            body = sub(body, fresh, 1)
            local = g + [(sub(body, 0), body)]
            prove([s, t], eq, local)
            prove(sub(template, s), branch, local)
            need(sub(template, t) == a)
        case ['i', branch]:
            need(len(a) == 1)
            body = sub(a[0], fresh + 1, 1)
            base = sub(body, 0)
            hyp = sub(body, fresh) if len(base) == 2 and base[0] == base[1] else base
            prove(sub(body, [0, fresh, 0]), branch, g + [([], hyp)])
        case _:
            need(False)
goal, proof = json.load(open(0))
prove(goal, proof, [([], [1, 1])])
print('ACCEPT')
```

References

[1] Vladimir Reshetnikov, *Concise presentations of intuitionistic primitive recursive arithmetic*, ProveIt repository, snapshot `11814771b7490dfbea0ed5fcf0f5b23586334764`. https://github.com/VladimirReshetnikov/ProveIt/tree/11814771b7490dfbea0ed5fcf0f5b23586334764/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] ProveIt, `Lean/ConciseIPRA/NativeDiagonal.lean`, especially `GPrf`, `GPrf.qInd`, and `gprf_iff_nprf`, same snapshot. A frozen copy is included.

[3] ProveIt, `Lean/ConciseIPRA/NativeClausalDiagonal.lean`, especially `KPrf` and `kprf_iff_nprf`, same snapshot.

[4] ProveIt, `Lean/ConciseIPRA/NativeBiInterpretation.lean` and `BiInterpretation.lean`, same snapshot: native/ordinary arithmetic interpretations, both composites, and consistency equivalence.

[5] ProveIt, `Research/raw_clausal/reference/foundations/`: `inherited_choice_basis.tex`, `inherited_ordinary_bridge.tex`, `scalar_predecessor.tex`, `equality_reflection.tex`, and `primitive_recursion.tex`, same snapshot.

[6] ProveIt, six archives in `docs/incoming/`, same snapshot. In particular `ipra_clause_discharge_2026-11-11.zip`, `ipra_inferred_witness_2026-11-11.zip`, and `ipra_deterministic_opening_2026-11-11.zip`. Exact comparison modules and file hashes are included.